

```

INIT_E =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_F => X"0000000000000000000000000000000000000000000000000000000000000000"
)
port map (
    RDATA => RDATA_c,
    RADDR => RADDR_c,
    RCLK => RCLK_c,
    RCLKE => RCLKE_c,
    RE => RE_c,
    WADDR => WADDR_c,
    WCLKN=> WCLKN_c,
    WCLKE => WCLKE_c,
    WDATA => WDATA_c,
    WE => WE_c
);

```

SB_RAM2048x2NRNW

SB_RAM2048x2NRNW // Negative edged Read and Write – i.e. RCLKN WRCKLN
(RDATA, RCLKN, RCLKE, RE, RADDR, WCLKN, WCLKE, WE, WADDR, MASK, WDATA);

```

SB_RAM2048x2NRNW ram2048x2nrnw_inst (
    .RDATA(RDATA_c[1:0]),
    .RADDR(RADDR_c[10:0]),
    .RCLKN(RCLKN_c),
    .RCLKE(RCLKE_c),
    .RE(RE_c),
    .WADDR(WADDR_c[10:0]),
    .WCLKN(WCLKN_c),
    .WCLKE(WCLKE_c),
    .WDATA(WDATA_c[1:0]),
    .WE(WE_c)
);
defparam ram2048x2nrnw_inst.INIT_0 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_1 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_2 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_3 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_4 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_5 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_6 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_7 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_8 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_9 =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_A =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_B =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst.INIT_C =
256'h0000000000000000000000000000000000000000000000000000000000000000;

```

```

defparam ram2048x2nrnw_inst .INIT_D =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst .INIT_E =
256'h0000000000000000000000000000000000000000000000000000000000000000;
defparam ram2048x2nrnw_inst .INIT_F =
256'h0000000000000000000000000000000000000000000000000000000000000000;

```

VHDL Instantiation:

```

ram2048x2nrnw_inst : SB_RAM2048x2NRNW
generic map (
INIT_0 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_1 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_2 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_3 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_4 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_5 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_6 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_7 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_8 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_9 =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_A =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_B =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_C =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_D =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_E =>
X"0000000000000000000000000000000000000000000000000000000000000000",
INIT_F => X"0000000000000000000000000000000000000000000000000000000000000000"
)
port map (
    RDATA => RDATA_C,
    RADDR => RADDR_C,
    RCLKN => RCLKN_C,
    RCLKE => RCLKE_C,
    RE => RE_C,
    WADDR => WADDR_C,
    WCLKN => WCLKN_C,
    WCLKE => WCLKE_C,
    WDATA => WDATA_C,
    WE => WE_C
);

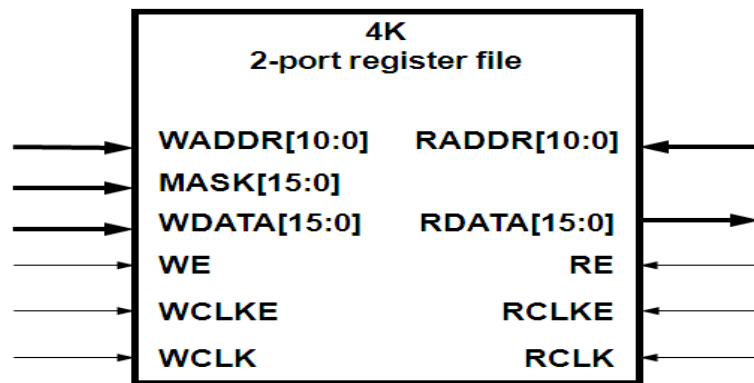
```

SB_RAM40_4K

SB_RAM40_4K is the basic physical RAM primitive which can be instantiated and configured to different depth and dataports. The SB_RAM40_4K block has a size of 4K bits with separate write and read ports, each with independent control signals. By default, input and output data is 16 bits wide, although the data width is configurable using the READ_MODE and WRITE_MODE parameters. The data contents of the SB_RAM40_4K block are optionally pre-loaded during ICE device configuration.

SB_RAM40_4K Naming Convention Rules

RAM Primitive Name	Description
SB_RAM40_4K	Posedge Read clock, Posedge Write clock
SB_RAM40_4KNR	Negedge Read clock, Posedge Write clock
SB_RAM40_4KNW	Posedge Read clock, Negedge Write clock
SB_RAM40_4KNRNW	Negedge Read clock, Negedge Write clock



The following table lists the signals for both ports.

SB_RAM40_4K RAM Port Signals		
Signal Name	Direction	Description
WDATA[15:0]	Input	Write Data input
MASK[15:0]*	Input	Bit-line Write Enable input, active low. Applicable only when WRITE_MODE parameter is set to 0.
WADDR[7:0]	Input	Write Address input. Selects up to 256 possible locations
WE	Input	Write Enable input, active high
WCLK	Input	Write Clock input, rising-edge active
WCLKE	Input	Write Clock Enable input
RDATA[15:0]	Output	Read Data output
RADDR[7:0]	Input	Read Address input. Selects one of 256 possible locations
RE	Input	Read Enable input, active high
RCLK	Input	Read Clock input, rising-edge active
RCLKE	Input	Read Clock Enable input

Parameter Name	Description	Parameter Value	Configuration
INIT_0,,INIT_F	RAM Initialization Data. Passed using 16 parameter strings, each comprising 256 bits. (16x256=4096 total bits)	INIT_0 to INIT_F	Initialize the RAM with predefined value
WRITE_MODE	Sets the RAM block write port configuration	0	256x16
		1	512x8
		2	1024x4
		3	2048x2
READ_MODE	Sets the RAM block read port configuration	0	256x16
		1	512x8
		2	1024x4
		3	2048x2

SB_RAM40_4K

Verilog Instantiation:

// Physical RAM Instance without Pre Initialization

```
SB_RAM40_4K ram40_4kinst_physical (
    .RDATA(RDATA),
    .RADDR(RADDR),
    .WADDR(WADDR),
    .MASK(MASK),
    .WDATA(WDATA),
    .RCLKE(RCLKE),
    .RCLK(RCLK),
    .RE(RE),
    .WCLKE(WCLKE),
    .WCLK(WCLK),
    .WE(WE)
);
defparam ram40_4kinst_physical.READ_MODE=0;
defparam ram40_4kinst_physical.WRITE_MODE=0;
```

VHDL Instantiation:

-- Physical RAM Instance without Pre Initialization

```
ram40_4kinst_physical : SB_RAM40_4K
generic map (
    READ_MODE => 0,
    WRITE_MODE => 0 )
port map (
    RDATA=>RDATA,
    RADDR=>RADDR,
    WADDR=>WADDR,
    MASK=>MASK,
    WDATA=>WDATA,
    RCLKE=>RCLKE,
    RCLK=>RCLK,
    RE=>RE,
    WCLKE=>WCLKE,
    WCLK=>WCLK,
    WE=>WE
);
```

SB_RAM40_4KNR

Verilog Instantiation:

// Physical RAM Instance without Pre Initialization

```
SB_RAM40_4KNR ram40_4knrinst_physical (
    .RDATA(RDATA),
    .RADDR(RADDR),
    .WADDR(WADDR),
    .MASK(MASK),
    .WDATA(WDATA),
    .RCLKE(RCLKE),
    .RCLKN(RCLKN),
    .RE(RE),
    .WCLKE(WCLKE),
    .WCLK(WCLK),
    .WE(WE)
);
defparam ram40_4knrinst_physical.READ_MODE=0;
defparam ram40_4knrinst_physical.WRITE_MODE=0;
```

VHDL Instantiation:

-- Physical RAM Instance without Pre Initialization

```
ram40_4knrinst_physical : SB_RAM40_4KNR
generic map (
    READ_MODE => 0,
    WRITE_MODE => 0
)
port map (
    RDATA=>RDATA,
    RADDR=>RADDR,
    WADDR=>WADDR,
    MASK=>MASK,
    WDATA=>WDATA,
    RCLKE=>RCLKE,
    RCLKN=>RCLKN,
    RE=>RE,
    WCLKE=>WCLKE,
    WCLK=>WCLK,
    WE=>WE
);
```

SB_RAM40_4KNW

Verilog Instantiation:

// Physical RAM Instance without Pre Initialization

```
SB_RAM40_4KNW ram40_4knwinst_physical (
    .RDATA(RDATA),
    .RADDR(RADDR),
    .WADDR(WADDR),
    .MASK(MASK),
    .WDATA(WDATA),
    .RCLKE(RCLKE),
    .RCLK(RCLK),
    .RE(RE),
    .WCLKE(WCLKE),
    .WCLKN(WCLKN),
    .WE(WE)
);
defparam ram40_4knwinst_physical.READ_MODE=0;
defparam ram40_4knwinst_physical.WRITE_MODE=0;
```

VHDL Instantiation:

-- Physical RAM Instance without Pre Initialization

```
ram40_4knwinst_physical : SB_RAM40_4KNW
generic map (
    READ_MODE => 0,
    WRITE_MODE => 0
)
port map (
    RDATA=>RDATA,
    RADDR=>RADDR,
    WADDR=>WADDR,
    MASK=>MASK,
    WDATA=>WDATA,
    RCLKE=>RCLKE,
    RCLK=>RCLK,
    RE=>RE,
    WCLKE=>WCLKE,
    WCLKN=>WCLKN,
    WE=>WE
);
```

SB_RAM40_4KNRNW

Verilog Instantiation:

// Physical RAM Instance without Pre Initialization

```
SB_RAM40_4KNRNW ram40_4knrnwinst_physical (
    .RDATA(RDATA),
    .RADDR(RADDR),
    .WADDR(WADDR),
    .MASK(MASK),
    .WDATA(WDATA),
    .RCLKE(RCLKE),
    .RCLKN(RCLKN),
    .RE(RE),
    .WCLKE(WCLKE),
    .WCLKN(WCLKN),
    .WE(WE)
);
defparam ram40_4knrnwinst_physical.READ_MODE=0;
defparam ram40_4knrnwinst_physical.WRITE_MODE=0;
```

VHDL Instantiation:

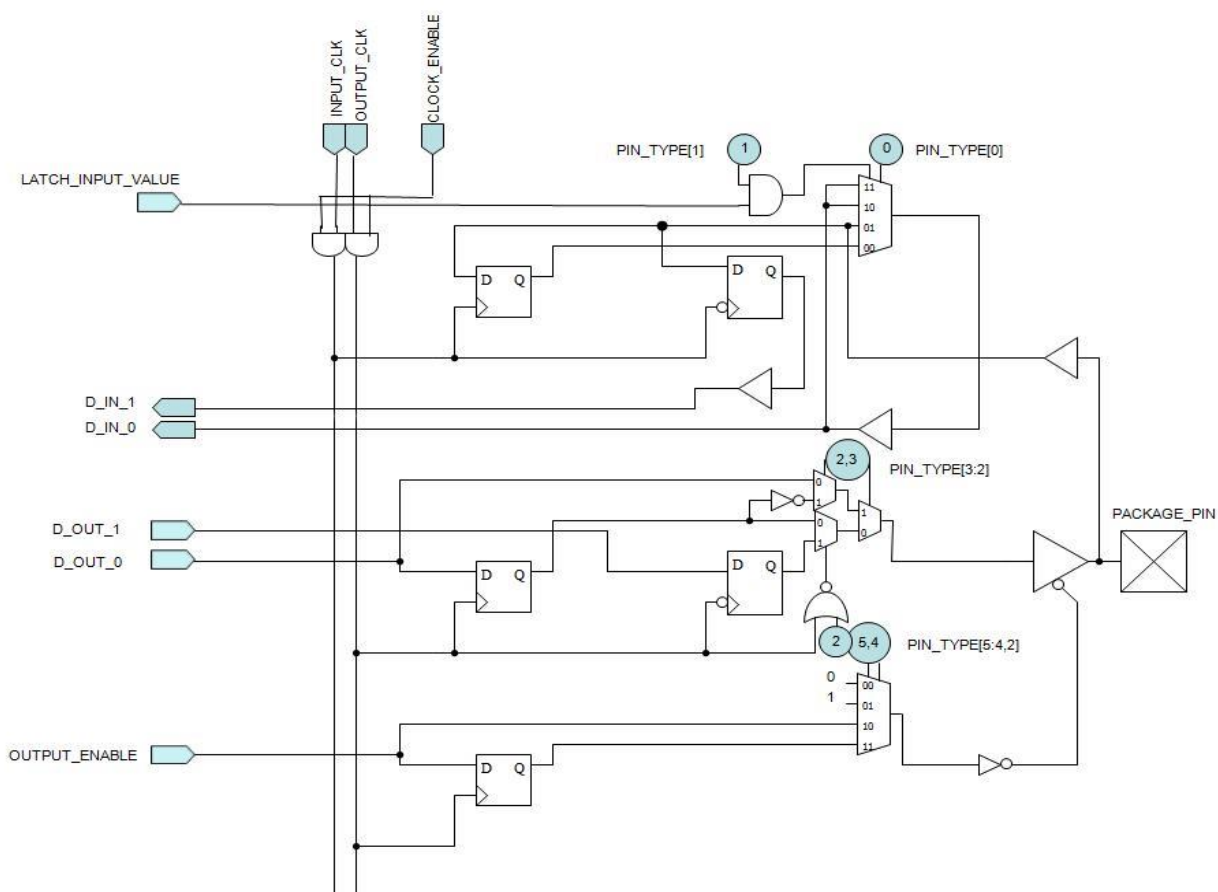
-- Physical RAM Instance without Pre Initialization

```
ram40_4knrnwinst_physical : SB_RAM40_4KNRNW
generic map (
    READ_MODE => 0,
    WRITE_MODE => 0
)
port map (
    RDATA=>RDATA,
    RADDR=>RADDR,
    WADDR=>WADDR,
    MASK=>MASK,
    WDATA=>WDATA,
    RCLKE=>RCLKE,
    RCLKN=>RCLKN,
    RE=>RE,
    WCLKE=>WCLKE,
    WCLKN=>WCLKN,
    WE=>WE
);
```

IO Primitives

SB_IO

The SB_IO block contains five registers. The following figure and Verilog template illustrate the complete user accessible logic diagram, and its Verilog instantiation.



Default Signal Values

The iCEcube2 software assigns the logic '0' value to all unconnected input ports except for CLOCK_ENABLE.

Note that explicitly connecting a logic '1' value to port CLOCK_ENABLE will result in a non-optimal implementation, since an extra LUT will be used to generate the Logic '1'. If the user's intention is to keep the Input and Output registers always enabled, it is recommended that port CLOCK_ENABLE be left unconnected.

Input and Output Pin Function Tables

Input and Output functions are independently selectable via PIN_TYPE [1:0] and PIN_TYPE [5:2] parameter settings respectively. Specific IO functions are defined by the combination of both attributes. This means that the complete number of combinations is 64, although some combinations are not valid and not defined below.

Note that the selection of IO Standards such as SSTL and LVCMOS are not defined by these tables.

Input Pin Function Table				
#	Pin Function Mnemonic	PIN_TYPE[1:0]		Functional Description of Package Pin Input Operation
1	PIN_INPUT	0	1	Simple input pin (D_IN_0)
2	PIN_INPUT_LATCH	1	1	Disables internal data changes on the physical input pin by latching the value.
3	PIN_INPUT_REGISTERED	0	0	Input data is registered in input cell
4	PIN_INPUT_REGISTERED_LATCH	1	0	Disables internal data changes on the physical input pin by latching the value on the input register
5	PIN_INPUT_DDR	0	0	Input 'DDR' data is clocked out on rising and falling clock edges. Use the D_IN_0 and D_IN_1 pins for DDR operation.

Output Pin Function table						
#	Pin Function Mnemonic	PIN_TYPE[5:2]				Functional Description of Package Pin Output Operation
1	PIN_NO_OUTPUT	0	0	0	0	Disables the output function
2	PIN_OUTPUT	0	1	1	0	Simple output pin, (no enable)
3	PIN_OUTPUT_TRISTATE	1	0	1	0	The output pin may be tristated using the enable
4	PIN_OUTPUT_ENABLE_REGISTERED	1	1	1	0	The output pin may be tristated using a registered enable signal
5	PIN_OUTPUT_REGISTERED	0	1	0	1	Output registered, (no enable)
6	PIN_OUTPUT_REGISTERED_ENABLE	1	0	0	1	Output registered with enable (enable is not registered)
7	PIN_OUTPUT_REGISTERED_ENABLE_REGISTERED	1	1	0	1	Output registered and enable registered
8	PIN_OUTPUT_DDR	0	1	0	0	Output 'DDR' data is clocked out on rising and falling clock edges
9	PIN_OUTPUT_DDR_ENABLE	1	0	0	0	Output data is clocked out on rising and falling clock edges
10	PIN_OUTPUT_DDR_ENABLE_REGISTERED	1	1	0	0	Output 'DDR' data with registered enable signal
11	PIN_OUTPUT_REGISTERED_INVERTED	0	1	1	1	Output registered signal is inverted
12	PIN_OUTPUT_REGISTERED_ENABLE_INVERTED	1	0	1	1	Output signal is registered and inverted, (no enable function)
13	PIN_OUTPUT_REGISTERED_ENABLE_REGISTERED_INVERTED	1	1	1	1	Output signal is registered and inverted, the enable/tristate control is registered.

Syntax Verilog Use

Output Pin Function is the bit vector associated with PIN_TYPE [5:2] and Input Pin Function is the bit vector associated with PIN_TYPE [1:0], resulting in a 6 bit value PIN_TYPE [5:0]

```
defparam my_generic_IO.PIN_TYPE = 6'b{Output Pin Function, Input Pin Function};
```

DDR IO Configuration

The following setting configures the SB_IO into a DDR IO.

```
defparam my_DDR_IO.PIN_TYPE = 6'b100000;  
  
// PIN_TYPE [5:2] = 1000  
// PIN_TYPE [1:0] = 00
```

This creates a DDR IO pin whereby the input data is clocked in on both the rising and falling input clock edges.

The output 'DDR' data is clocked out on rising and falling output clock edges, and the output may be tri-stated, using the output enable port of the SB_IO.

High Drive SB_IO

IO's in iCE40/iCE40LM device can be configured with different drive strengths to increase the IO output current. To configure an SB_IO with specific drive value, the user needs to specify the "DRIVE_STRENGTH" synthesis attribute on the SB_IO instance and the IO should be configured as output-only registered IO.

Synthesis Attribute Syntax:

```
/* synthesis DRIVE_STRENGTH = <Drive value> */
```

Drive Value:

Drive Strength Value	Description.
x1	Default drive strength. No replication of SB_IO.
x2	Increase default drive strength by 2. SB_IO replicated once.
x3	Increase default drive strength by 3. SB_IO replicated twice.

Note: High drive SB_IO is available only in selected ICE40/ICE40LM packages. Refer to Chapter 12 in iCEcube2_userguide for the list of supported device packages.